

WAFT: The Evolutionary Code Laboratory

SUMMARY

The Physics is the Scint System, which acts as a fitness function through Reality Fracture Detection.

Generated: 2026-01-11 23:04 UTC | Edition: Black & White | Ideas: 13

What is WAFT?

WAFT is a Python framework for directed evolution of self-modifying AI agents. Think of it as an operating system for AI agent research projects. Instead of just building agents that execute code, WAFT enables you to breed agents that can modify their own code, evolve through mutations, and be tested in fitness systems with complete lineage tracking for scientific research.

The Three Pillars

THE SUBSTRATE

Agents write their own Python source code. In WAFT, code is DNA. Each agent has a unique genome ID (SHA-256 hash of their code and configuration). Agents can spawn variants with mutations, evolve by hot-swapping their own code, and reproduce by creating children with specific genetic modifications. This enables true self-modification where agents improve themselves.

THE PHYSICS

The Scint System acts as a fitness function through Reality Fracture Detection. This serves as natural selection that eliminates weak mutations. Agents face quests testing their ability to handle four error types: SYNTAX_TEAR (formatting errors), LOGIC_FRACTURE (math errors and contradictions), SAFETY_VOID (harmful content), and HALLUCINATION (fabricated facts). Agents must stabilize these errors to survive, with fitness measured by stability, efficiency, and safety scores.

THE FLIGHT RECORDER

A rigorous telemetry system for generating phylogenetic trees of agent lineage. Every evolutionary action is recorded with complete context: genome ID, parent ID, generation number, event type, payload with full context, and fitness metrics. This enables reconstruction of complete family trees for scientific publication, allowing phylogenetic analysis, mutation impact measurement, fitness landscape mapping, and convergence analysis.

How It Works

WAFT provides project scaffolding through a unified CLI interface. Run one command to create a fully configured project with best practices built in. The system uses uv for fast Python package management, creates a _pyrite memory structure for organizing project knowledge, includes CI/CD pipelines ready to go, and provides optional AI agent templates. Everything is file-based with no database, no server, just plain text files that work with git.

QUICK START

Install: `uv tool install waft`

Create: `waft new my_lab`

Verify: `waft verify`

What Makes WAFT Unique: WAFT is ambient, working quietly in the background without getting in your way. It is self-modifying, allowing projects to evolve their own structure over time. It is a meta-framework that orchestrates existing tools rather than replacing them. Everything is file-based, making it git-friendly and portable. The system includes gamification with D&D-style progression, epistemic tracking, and scientific observation with complete lineage tracking.

Key Features

- | **Self-Modification:** Agents can modify their own code and evolve
- | **Fitness Testing:** Scint Gym evaluates agents through error handling
- | **Lineage Tracking:** Complete phylogenetic trees for scientific analysis
- | **File-Based:** Everything is plain text, git-friendly, no database
- | **Scientific Instrument:** Produces data for research on AI cognition